MNXB11-project

Hanna Palm, Jarl Olsson, Yoline Ruiter

October 2026



LUND
UNIVERSITY

1 Introduction

For this project we were given temperature datasets from SMHI, the Swedish Meteorological and Hydrological Institute. The data included information about the location, date, hour, temperature and quality of the measurements. With this data we aimed to investigate three different hypotheses about the temperature in Sweden using the programming skills gained throughout the course.
The first subject of observation is the coldest day per month in the locations Lund, Karlstad, Uppsala and Halmstad. No systematic observations are expected to be found.
The second being whether there is any systemic difference of the temperature between the days of the week, which we expect there not to be.
The final being investigating the average temperature in Uppsala with the long dataset between 1722-2022, which is expected to show evidence of climate change in the form of increasing average temperatures over the last century due to anthropogenic greenhouse gas emissions.
The collaborative development of the code was done with git following policy A with Jarl being the release manager handling the upstream with Hanna and Yoline working on their own forks and making pull requests to the upstream.

2 Method

2.1 Data Preparation

The Uppsala temperature series dataset required no data preparation by us due to already being extensively reconstructed and corrected by Hans Bergström and Anders Moberg [1]. We decided to use the temperature data that was corrected for urban effects and use every single datapoint including the ones from other nearby locations and interpolated data.
The datasets for Lund, Uppsala, Halmstad and Karlstad covering the period from 2020 to 2025 were generated by opening the corresponding data from the project template in VScode. The six years of data were selected and put in four separate files along with a header showing the type of data featured.[19]

2.2 Data Extraction

2.2.1 DataPoint Class

As an intermediate before analysis we decided to store each data point as a custom class called DataPoint which inherits from TObject which was then stored inside a TTree in a root file in a very similar way to Homework 6. Before being able to put the data in this format we needed to be able to extract it from the .csv/.dat files.

The `DataPoint` class represents one temperature measurement. It has three constructors to accommodate for different data formats. A `DataPoint` object stores the following information about the measurement:

- Date of measurement
- Time of measurement (in hours)
- Temperature
- Quality of measurement

Subsequently, there are two private functions `_split_date()` and `_set_day_of_week()` which adds additional information to the `DataPoint` object. The `_split_date()` function splits the date and stores the day, month and year of measurement while `_set_day_of_week()` adds the day of the week. The `_set_day_of_week()` function uses the external library `date.h` to find out the day of the week based on the date of the measurement.

2.2.2 Csv2TTree

The CSV to ROOT conversion functionality was implemented in the `Csv2TTree` module. It contains three functions: `parse_csv_file()`, `to_root()` and `csv_to_root()`.

The `parse_csv_file()` function takes the path to a .csv file as input and returns a vector of `DataPoint` objects. It iterates through each line in the given .csv file. Since some data files contains metadata before the temperature data begins, it iterates over lines until a specific header is found which functions as an identifier for the start of the temperature measurements.

Once the header has been found, each temperature measurement is converted into a `DataPoint` object and stored an a vector. Having parsed through the entire file, the function then returns a vector containing the extracted data points.

The `to_root()` function takes a vector of `DataPoint` objects and an output filename as input. It creates a ROOT file using the RECREATE option which either creates a new file or overwrites an existing file with the same name. The output file is stored in the `datasets/` directory.

Next, the function creates a `TTree` containing a `datapoint` branch which stores information for each `DataPoint` object in the given vector of `DataPoint` objects. In the end, the tree is written to the ROOT file.

The `csv_to_root()` function carries out the entire .csv to .root file conversion by first calling `parse_csv_file()` to extract the measurements and then passes the resulting vector to `to_root()` to store the data as a ROOT file.

2.2.3 Dat2TTree

To get the data from the .dat file was quite straightforward using a similar approach to the .csv methodology with the caveat of only using one function to go directly from .dat to .root. Each line was gathered from the .dat file and then a string stream was used to split up the columns into different variables after converting the string variables for the year, month, day to integers and corrected temperature to a double. The date was created to match the format from the csv file and finally the date and the temperature was put into the setter for the DataPoint class, with -1 for the hour to indicate that it is not valid for this dataset and the boolean quality set to true since all the data points were deemed valid in the data preparation phase.

2.3 Analysis

2.3.1 Climate Change

The climate change analysis is split into four functions, the first `climate_change()` takes the path of the root file with data generated by Dat2TTree and opens it, retrieves the data in the form of a TTree and calls the plotting function `plot_average_temperature()` and closes the root file.

The plotting function uses multiple TGraphs on one TCanvas for the different elements of the plot, that being the red points which are years with an average temperature above the overall average temperature over the years of the dataset, the blue points below the overall average, a black reference line with constant value corresponding to the overall average. In addition to this there is also a 10 year and 20 year moving average.

The next step in the plotting function is creating a vector of doubles with the yearly average values. This is done with a loop that calls the `average_temperature()` function for each year in the dataset and putting the value in the vector. This vector is then used to calculate the overall average using the functions `std::accumulate()` and the vector function `size()`

The `average_temperature()` function loops through each entry in the TTree with DataPoint value year belonging to the specified year and calculates the average temperature of these entries.

The `moving_average()` function takes the vector of yearly average temperatures, the year to calculate the moving average around, the size of the moving average window which must be even e.g 10,20 and the first and last year of the data. After validating the inputs, the average over the window is calculated with a loop between the years in the range from year-N/2 to year+N/2-1 inclusively, which means that the average is actually centered on year-1/2 and is plotted with this offset in mind.

Finally, in the plotting function a loop over each year fills the TGraphs with a series of if-statements to put the average temperatures into the cold/hot and to make sure only valid years get used to call the moving average function. After this there is a lot of root commands to make the plot which is saved with the specified path/name.

2.3.2 Weekday

The `Weekday` module consists of six functions which handles the analysis and visualization of temperature with respect to the day of the week.

There are two filter functions: `year_filter` and `weekend_filter`.

The `year_filter` function takes in a vector of `DataPoint` objects along with a year and returns a vector of `DataPoint` objects corresponding to measurements only taken during the given year. It also discards any measurements that are not marked as being of good quality (G in the raw CSV data files).

The `weekend_filter` function takes in a vector of `DataPoint` objects and returns a vector of `DataPoint` objects corresponding to measurements only taken on Saturdays and Sundays, i.e. during weekends.

The `average_temp_by_weekday()` function calculates the average temperature for each day of the week, given a vector of `DataPoint` objects. It returns a `std::array>double, 7>`, where each element represents the average temperature for a weekday, ordered from Monday to Sunday.

The last three functions handles visualization of the analyzed data.

The `plot_average_temp_by_weekday()` creates a bar chart with the average temperature for each day of the week, given the path to a csv file, a name for the output figure and a year as input. This is done in several steps. First, the function calls `parse_csv_file()` to load the data from the given csv file. Using the resulting `DataPoint` vector as input, `year_filter()` is applied. This results in an array of `DataPoint` objects with measurements taken within one given year. Next, `average_temp_by_weekday()` is called, using the filtered `DataPoint` vector as input.

Finally a ROOT histogram is created with seven bins, corresponding to each day of the week. The calculated average temperatures for each day of the week are assigned to the corresponding bins. The resulting plot is saved as a PNG image in the `images/` directory.

The `plot_av_temp_weekday_comparison()` function extends the previously mentioned `plot_average_temp_by_weekday()` by comparing average weekday temperatures across several years. It is called in the same way as the previous plotting function except it accepts a vector of years where `plot_average_temp_by_weekday()` only takes a single year.

For each given year, the function filters the original dataset, calculates the average temperature for each weekday, and creates a separate histogram. Each histogram is added to a `THStack`, which is later drawn using the `NOSTACK` option. The `NOSTACK` option ensures that the bars are displayed side by side rather than stacked on top of each other.

To easily distinguish between the years, one color is assigned to each histogram. The final plot is saved as a PNG image in the `images/`directory.

The `plot_weekday_temp_trends()` function works very similar to `plot_av_temp_weekday_comparison()` except it creates a line plot instead to allow for better visualization over longer periods of time. It also calculates the mean and standard deviation for each year's data and prints the information to the terminal.

2.3.3 Monthly coldest day

The monthly coldest day analysis consists out of two sets of code. These are `monthly_coldest.cxx` and `allcities.cxx`. The `monthly_coldest.cxx` creates a scatter plot showing the calender day that is coldest for every month. The graph features an average, a mean value of the average and a standard deviation. The taken timeline of the plot is 6 years; from 2020 until 2025. The code is used for producing this plot for the cities Lund, Uppsala, Halmstad and Karlstad. Since this code is used for four different cities, main contains a loop that processes each city. The second set of code is `allcities.cxx`. This piece

of code creates an empty canvas which is then filled by the four generated PNG's in a two by two fashion. In addition, the timestamp is printed on the resulting `all_cities_2x2.png` allowing for enhanced file management.

The `monthly_coldest.cxx` consists out of several distinguishable parts. First, the input data is read. In the main the `CSV_2_TREE.cxx` code is used to convert the csv code into a root file. This root file is then opened and read and the data tree is retrieved. The input file has data stored in several branches. The code connects the datapoint branch to a DataPoint pointer and reads the DataPoint object via the datapoint branch. This will allow for reading the data using `datapoint->get_date();` for for example the date. `sscanf(currentDate.c_str()` is then used for extracting the year, month and day from the date string from DataPoint. The coldest date is then stored in a map. Which is accessed later on. In addition, an outpoint object is created that will store the relevant data later on in the same formatting as the input data.

To find the lowest temperature of each month, and storing the corresponding date for all valid data, an initial value is set which is replaced when a colder temperature is found. If no colder temperature is found the coldest temperature is stored in a map. This map `coldest` uses the key `<year,month>` with the variable Colestinfo. The coldest data is continuously stored and replaced if a lower temperature is found. To do this, the datapoint pointer is then used to get the data from the tree.

In the next part, 12 monthfiles are created with each a corresponding month tree and each tree having an output branch. The next part of code then takes the date and temperature data from the map and stores it in the tree. These files are not used later on in the plotting but they are used to print the coldest data and temperature in the terminal. This is done since it was no initial part of the scatterplot but generally interesting to know.

The canvas and the graphs are consequently created. Each year gets their own graph and color but they will all be plotted on the same canvas. The map `coldest` is used to get the month and the corresponding resulting day on which the temperature is lowest.

After that we move on to the calculations of the average day on which the temperature was lowest. From this data, a mean of the average day and standard deviation was calculated.

Consequently, the average plot is set up the same as the other plots. After which the plots are filled and added to the legend, including the average data. Each plot gets their own title corresponding to the location which is used when looping over the code four times. This is done by recognizing the name in the title of the root file. The graph is then drawn with both yeargraphs and the averagegraph after which it is saved under the name of the corresponding location.

After looping over this four times four PNG's have been created. These images are then used as input by the `allcities.cxx` after which an image is generated which features the images in a two by two format. To make the code, sources [1] to [20] where used.

2.4 SLURM

Before running the project a SLURM job, the time and memory allocation was profiled using the same method as done in Homework 3. After updating the resource allocations it was still not running without errors, with many referring to conflicts with the Shell variable XAUTHORITY which seems to be used in the GUI a spects of the course container. By unsetting/deleting the variable before running the runproject script, the errors went away.

3 Results

3.1 Coldest day of the months

The results obtained by the monthly coldest analysis are shown in figure 1 . Since no repetition nor correlation is expected due to the randomness of the cold and the weather not keeping track of the months, complete randomness and thus an overall average of day 15 was expected. For all cases the mean was found around the 15th day, supporting this hypothesis. However, from the calculated average values we can see a relative clear trend. First, the average day is about 10 - 20, followed by a decrease to 4 - 10 after which the average dat is yet increased to around 15 - 25.

In December, the years are quite similar in distribution; 2022, 2023 and 2024 are continuously below the average, while the other months stay above the average day. This indicates a harsher colder winters. This trend was not seen for 2025, therefore no future predictions can be made.

For January we see a trend occurring where the coldest day of 2025 is in all cases below the average. We see this same trend for 2020 and 2022. The 2021 data is shown to repetitively be shown high above the average, around the end of the month, except for the city located most south of the four; LUND, where the coldest day was found to occur on the 16th of November. The observed trend for 2020 and 2021 does not show any ambition for future prediction.

In February, not a lot of patterns are visible except for 2021 being below average by repetition.

March shows a repetition of 2023 and 2024 (and 2025) being below the average. This could indicate a continuous trend, however in 2025 the coldest day was 5 days above average. Thus not showing a reliable trend.

April, May and June all show a relatively low index of the coldest day. This might indicate that, even though it is generally warmer those months, the weather has a way of repeating itself in a non random fashion. This repetitively low average for al four different locations might have potential for future predictions. It could possibly be explained by the days getting longer during that period. Also, the longest day is on June 20th making this a plausible argument that is observed thoughout the data. However, since we use a 6 year database this is not a reliable conlusion but merely an observation.

The expected randomness is a bit regained in July and August, which has average a slightly above day 15. However it is a bit above 15 in all cases. Even though this might seem as non-randomness, we do have to keep in mind that the standard deviation of the average day on which the temperature is coldest. This standard deviation is around 5 to 7 days. That is roughly 1/5 of a month.

September does not show a lot of correlations. Except for 2023 being continuously below average. October and November show a continuous increase where the coldest day of November is higher than October in all places.

The four cities where chosen to be able to compare geographical positions. Halmstad and Lund are roughly on the same latitude and Karlstad and Uppsala are approximately on the same latitude. Therefore the plots diagonal to each other can be compared. The plot of Halmstad is most smooth of all. It clearly shows the lower average over May and June and the higher average around the colder months. If this is now compared to Lund we see a decrease from March to July but more irregular winter/autum data.

When comparing Uppsala and Karlstad we see a constant decrease of the average from April to June. But more less predictable data overall. Since these places are generally colder it is expected.

Whether this data leads to be able to predict the coldest days in future years is very debatable since climate change has a big role in this. If we compare 2020 to 2025 we see more rough correlation than with the other years, thus disregarding the possibility of a correlation. In this same period (2020 - 2023), the corona pandemic began. Because of this transportation came to a halt causing a decrease in global warming. However, since Sweden is less populated (25 people per square km) [23] than e.g. the Netherlands where 539 people live per square km [22], the effect is expected to be very minimal or zero.

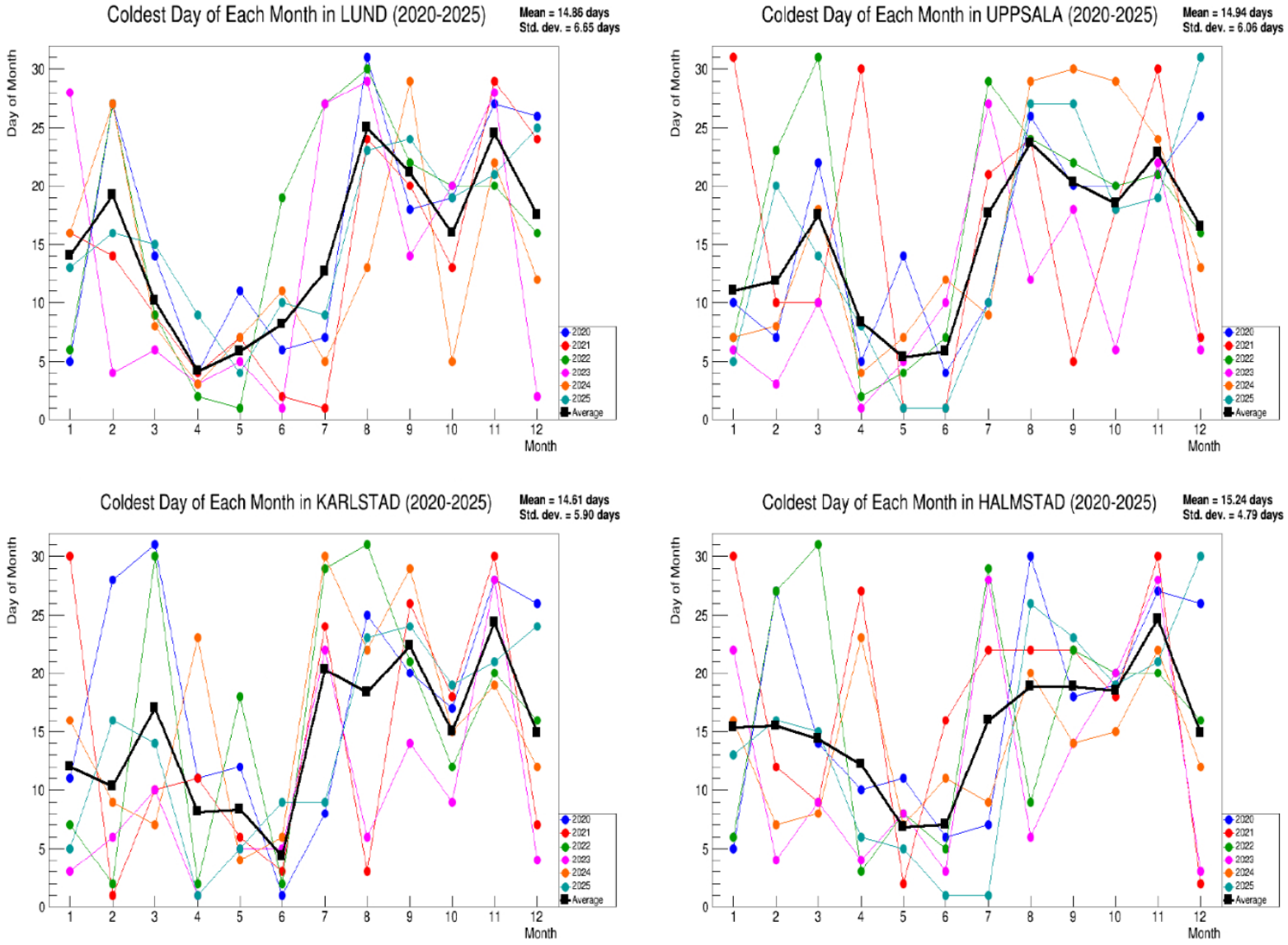


Figure 1: Monthly coldest days of 2020- 2025 in Lund, Uppsala, Karlstad and Halmstad

3.2 Climate Change

Figure 2 shows the calculated annual average temperatures in Uppsala in the years between 1722-2022 together with an overall average and moving averages. This analysis shows the prevalence of considerable fluctuations in the temperatures around the overall mean during the 18th and 19th centuries. However, from the early 20th century and onward there is a clear growing trend especially noticeable in the moving averages.

This analysis shows evidence of warming in the last few decades far larger than any fluctuations in the pre-industrial era of this dataset.

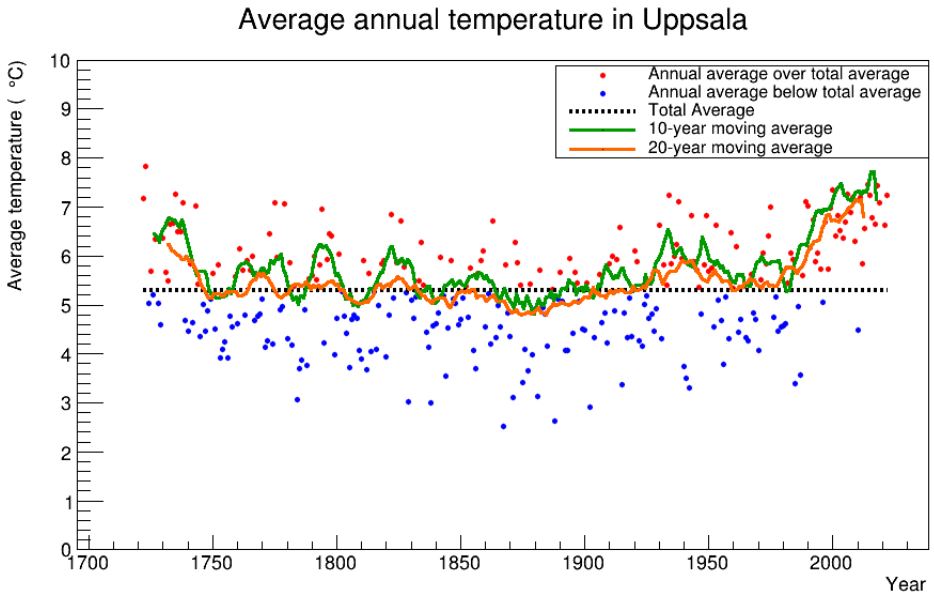


Figure 2: Average annual temperatures between 1722-2022 in Uppsala with years above the mean shown with the dashed black line in red and below in blue. In addition to the yearly averages there is 10 and 20 year moving averages in green and orange respectively. In the last few decades there is a clear increasing trend.

3.3 Average temperature by weekday

Figure 3 shows the average temperature for each day of the week. The aim of this analysis was to show whether temperatures follows a weekly pattern. While some smaller differences between the days within the same year does occur, it is not enough to show that there exists a trend.

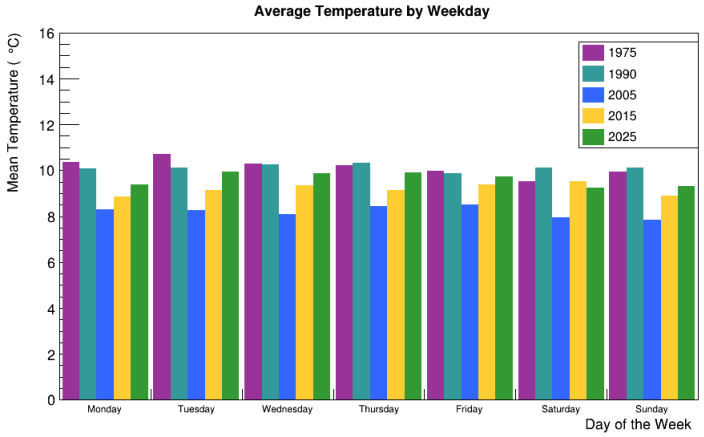


Figure 3: Average temperature for each day of the week in Lund.

Figure 4 shows how the average weekday temperatures have changed over a selected period of 20 years. An slight overall upward trend can be seen.

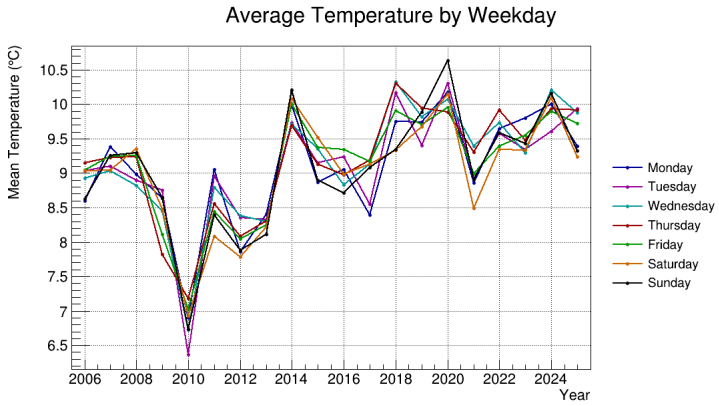


Figure 4: Temperature trends for each day of the week in Lund

4 Discussion

The annual mean temperature analysis shows clear evidence of increasing temperatures in Uppsala during the last century, which are very likely related to the global warming caused by anthropogenic greenhouse gas emissions. The weight of this analysis could be improved by doing the same analysis for more locations and seeing if the trends persist, this would only

require slight changes to make the plotting function more modular and access to more high quality datasets.

The week day analysis showed that there is no obvious relation between the day of the week and the average temperature as expected. I.e there is no weekday with a higher average temperature than another. In addition to this expected result we ended up seeing an increasing trend of the average temperature supporting the result from the climate change analysis of Uppsala with this data taken from Lund. However, further and more detailed analysis would be required to definitively claim there is no relation at all between the temperature of the weekdays.

The analysis of the coldest day per month could have been analyzed to a greater extent if more years and places where taken into account. In this case that was not done because of readability. If this was implemented, merely the amount of loops would have to be adapted. Expanded datasets of places with the same latitude or longitude could have been taken to improve the current observations since four independent cities are not enough to form a conclussion. In addition the coldest temperatures could have been plotted if a 3D plot or datalables where used. The code finding the coldest day per month could have been shortened by not using the 12 monthfiles. However since the coldest temperatures where not plotted it is still considered relevant information.

5 Conclusion

The analysis of the climate in Uppsala showed what we expected in the way of an increasing trend in the average annual temperatures over the last century. Additionally, analysis concerning the weekdays also hinted that this trend exists in Lund as well possibly warranting concluding that Sweden in general is experiencing a warming climate. This analysis showed what it was intended to do which was to show that there does not seem to be any connection between the days of the week and the temperature

The monthly coldest day shows a non random distribution over the months April to July. However, this observation needs to be nuanced since the standard deviation is found to be around 5 to 6 days. The mean of the average temperature is found to be around 15 days as is expected of a random distribution for a month. No clear increase or decrease was found when comparing 2020 data of all four places with 2025 data. The places similar in latitude show a increase of the decrease when moving further south. Similarities in places where the latitude was the same were not clearly found. Therefore the hypothesis was partially right. The decrease in average day for the months April to July was not foreseen but it could be explained by the days getting longer.

References

[1] H. Bergström and A. Moberg. “Daily Air Temperature and Pressure Series for Uppsala (1722–1998)”. In: *Climate Change* 53 (2002), pp. 213–252.

[2] cplusplus.com. *C++ cmath Library Reference*. URL: https://cplusplus.com/reference/cmath/ (visited on 10/28/2026).

[3] GeeksforGeeks. *C Pointers*. 2026. URL: https://www.geeksforgeeks.org/c/c-pointers/ (visited on 10/28/2026).

[4] GeeksforGeeks. *C++ Arrays*. URL: https://www.geeksforgeeks.org/cpp/cpp-arrays/ (visited on 10/28/2026).

[5] GeeksforGeeks. *Map in C++ Standard Template Library (STL)*. URL: https://www.geeksforgeeks.org/cpp/map-associative-containers-the-c-standard-template-library-stl/ (visited on 10/28/2026).

[6] GeeksforGeeks. *string::npos in C++ with Examples*. URL: https://www.geeksforgeeks.org/cpp/stringnpos-in-c-with-examples/ (visited on 10/28/2026).

[7] GeeksforGeeks. *Utility Library in C++*. URL: https://www.geeksforgeeks.org/cpp/utility-in-c/ (visited on 10/28/2026).

[8] OpenAI. *ChatGPT*. https://chatgpt.com/. Used for code optimisation, error resolution, and source formatting. Accessed 24–29 October 2026. 2026.

[9] Florido Paganelli. *MNXB11 2026 Help Manual*. https://canvas.education.lu.se. Accessed 28 October 2026. 2026.

[10] Florido Paganelli et al. *Lecture Slides MNXB11 1–7*. https://canvas.education.lu.se. Accessed 28 October 2026. 2026.

[11] Programiz. *C++ sscanf() Function*. URL: https://www.programiz.com/cpp-programming/library-function/cstdio/sscanf (visited on 10/28/2026).

[12] ROOT Team. *ROOT TAxis Class Reference*. URL: https://root.cern.ch/doc/master/classTAxis.html#a3b5c3a2189978931f1af6a84f868192a (visited on 10/28/2026).

[13] ROOT Team. *ROOT TDatime Class Reference*. URL: https://root.cern.ch/doc/master/classTDatime.html (visited on 10/28/2026).

[14] ROOT Team. *ROOT TGraph Class Reference*. 2026. URL: https://root.cern.ch/doc/master/classTGraph.html (visited on 10/28/2026).

[15] ROOT Team. *ROOT TH1F Class Reference*. URL: https://root.cern.ch/doc/master/classTH1F.html (visited on 10/28/2026).

[16] ROOT Team. *ROOT TLatex Class Reference*. URL: https://root.cern.ch/doc/master/classTLatex.html (visited on 10/28/2026).

[17] ROOT Team. *ROOT TPad Class Reference*. URL: https://root.cern.ch/doc/master/classTPad.html (visited on 10/28/2026).

[18] ROOT Team. *ROOT TTree Class Reference*. 2026. URL: https://root.cern.ch/doc/master/classTTree.html (visited on 10/28/2026).

[19] Oxana Smirnova and Florido Paganelli. *MNXB11 Project Template Repository*. 2026. URL: https://github.com/0xana/MNXB11-project-template (visited on 10/19/2026).

[20] W3Schools. *C++ Auto Keyword*. URL: https://www.w3schools.com/cpp/cpp_auto.asp (visited on 10/28/2026).

[21] W3Schools. *C++ Structures*. URL: https://www.w3schools.com/cpp/cpp_structs.asp (visited on 10/28/2026).

[22] Worldometer. *Netherlands Population*. URL: https://www.worldometers.info/world-population/netherlands-population/ (visited on 10/28/2026).

[23] Worldometer. *Sweden Population*. URL: https://www.worldometers.info/world-population/sweden-population/ (visited on 10/28/2026).

A Additional Code

The link to the Github repository: https://github.com/JarlLu/MNXB11-project.git